

Protocol Audit Report

Version 1.0

Sir0xGhanem

May 23, 2026

Prepared by: Mostafa Ghanem

Lead Auditor(s): - Mostafa Ghanem

Table of Contents

- Protocol Summary
 - Disclaimer
 - Risk Classification
 - Audit Details
 - Scope
 - Roles
 - Executive Summary
 - Issues Found
 - Findings
 - High
 - Medium
 - Low
 - Informational
 - Gas
-

Protocol Summary

PasswordStore is a protocol designed for storing and retrieving a user's password.

It is intended for single-user use only. Only the contract owner should be able to set and access the stored password.

Disclaimer

Mostafa Ghanem makes every effort to identify as many vulnerabilities as possible within the given time period. However, no responsibility is assumed for the findings provided in this document.

This security audit is not an endorsement of the underlying business logic or product. The review was time-boxed and focused solely on the security aspects of the Solidity implementation.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine issue severity.

Audit Details

Scope Details

- Commit Hash: `7d55682ddc4301a7b13ae9413095feffd9924566`

In Scope

1	
2	<code>./src/</code>
3	<code>PasswordStore.sol</code>

- Solidity Version: 0.8.18
 - Deployment Chain(s): Ethereum
-

Roles

- **Owner:** The user authorized to set and retrieve the password.
 - **Others:** No other users should be able to set or access the password.
-

Executive Summary

Issues Found

Severity	Count
High	2
Medium	0
Low	1
Info	1
Gas	0
Total	4

Findings

High Severity

[H-1] Storing the password on-chain makes it publicly readable

Description:

All data stored on-chain is publicly accessible. The `PasswordStore::s_password` variable is intended to be private and only retrievable via `PasswordStore::getPassword`, which is restricted to the contract owner.

However, this is a misunderstanding of blockchain privacy guarantees.

Anyone can directly read contract storage without interacting with the contract.

Impact:

The password is publicly exposed, completely breaking the intended privacy model of the protocol.

Proof of Concept:

1. Start a local blockchain:

```
1 make anvil
```

2. Deploy the contract:

```
1 make deploy
```

3. Read storage slot directly:

We use slot 1 because it stores `s_password`.

```
1 cast storage <CONTRACT_ADDRESS> 1 --rpc-url http://127.0.0.1:8545
```

Example output:

```
1 0x6d7950617373776f726400000000000000000000000000000000000000000014
```

4. Decode it:

[illegible]

Result:

```
1 myPassword
```

Recommended Mitigation:

- Do not store sensitive data on-chain in plaintext.
- Store only encrypted values, and perform decryption off-chain.
- Reconsider whether a `getPassword` function should exist at all, as it may leak sensitive workflow assumptions.

[H-2] Missing access control in setPassword

Description: The function `PasswordStore::setPassword` is declared `external`, but there is no access control restricting it to the owner.

Although NatSpec comments indicate only the owner should set the password, this is not enforced in code.

Impact: Any user can overwrite the stored password, breaking contract integrity.

Proof of Concept:

```
1 function test_anyone_can_set_password(address randomAddress) public {
2     vm.assume(randomAddress != owner);
3
4     vm.prank(randomAddress);
5
6     string memory expectedPassword = "myNewPassword";
7     passwordStore.setPassword(expectedPassword);
8 }
```

```
8
9     vm.prank(owner);
10
11     string memory actualPassword = passwordStore.getPassword();
12
13     assertEq(actualPassword, expectedPassword);
14 }
```

Recommended Mitigation:

Add an access control check:

```
1  if (msg.sender != s_owner) {
2      revert PasswordStore_NotOwner();
3  }
```

Informational

[I-1] Incorrect NatSpec documentation for getPassword

Description: The NatSpec for `PasswordStore::getPassword` incorrectly includes a parameter that does not exist:

```
1  @param newPassword The new password to set.
```

However, the function signature is:

```
1  getPassword()
```

Impact: This may mislead developers and auditors reviewing the contract.

Recommended Mitigation:

Remove the incorrect documentation line:

```
1  - * @param newPassword The new password to set.
```